

MPP
PRÁCTICA 3

**Implementación de un algoritmo evolutivo usando
Paralelismo Híbrido (OpenMP + MPI) así como su
Modelado y Auto-Optimización**

Curso 2025/26

José Luis García Valverde, subgrupo 1.1

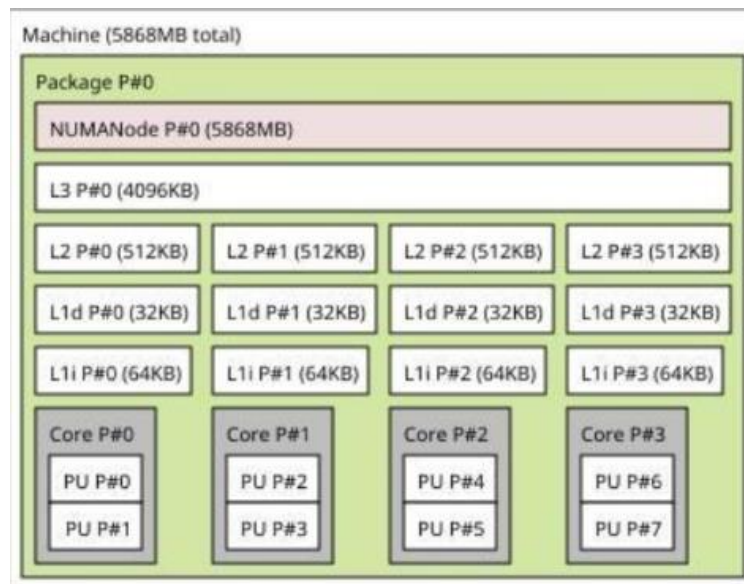
Eric Montalbán Atencia, subgrupo 1.3

Índice

Introducción.....	3
Cuestión 1.....	4
Cuestión 2.....	4
Cuestión 3.....	9
3.1 Obtención de datos experimentales	9
3.2 Instalación de la Rutina (bucle for) con 1 Nivel de Paralelismo	13
3.3 Ejecución	13
Cuestión 4. Modelo de Tiempo en Memoria Compartida con 2 niveles de paralelismo	20
Cuestión 5. Modelo de Tiempo en Memoria Distribuida	28
Cuestión 6. Conclusiones generales y valoración personal.....	33

Introducción

Lo primero de todo es mostrar la información del sistema computacional en el que se han realizado las pruebas, para ello y como en las anteriores prácticas, haremos uso del comando: `lstopo -f -p --no-legend --no-io sistema.svg`



Cuestión 1

Con el fin de obtener un código más eficiente en la parte de memoria compartida y poder maximizar el proceso de mejora de la población de individuos: reemplazar las llamadas a la función rand por rand_r (que es “thread safe”) para evitar, por un lado, que se genere la misma secuencia de valores aleatorios en diferentes hilos y, por otro, que se pierda paralelismo durante la generación de dichos valores debido a la secuencialidad que introduce la función rand.

Como ya se realizó en la práctica 1 obviamos la explicación para no ser redundantes.

Cuestión 2

Implementar un algoritmo híbrido, tomando como referencia las directivas OpenMP y primitivas MPI que arrojaron mejores resultados en las prácticas 1 y 2. Se deberán utilizar, por tanto, las funciones que se considere más adecuadas para obtener un código optimizado y que permita obtener las mejores prestaciones. Ejecutar el código variando el número de procesos y de hilos, y comparar los resultados obtenidos respecto a la versión secuencial utilizando las métricas de rendimiento de la Tabla 1.

Además, se deberá analizar la mejora obtenida en las prestaciones, considerando solo tiempo de ejecución y tiempo de ejecución + fitness (ver apartado (1) de Observaciones), respecto al algoritmo secuencial y también respecto a la mejor versión paralela obtenida en las prácticas anteriores usando el mismo sistema computacional para todas las versiones.

Antes de realizar la comparación entre las distintas implementaciones del algoritmo evolutivo, debemos primero saber cuál es la mejor configuración de procesos e hilos para la versión híbrida. Para ello hemos creado un script que ejecutará tres veces cada combinación de procesos e hilos y tomará la media del tiempo de ejecución y el fitness. Además, calculará la eficiencia $I = \text{fitness} / \text{tiempo de ejecución}$.

Estas son las distintas combinaciones de procesos e hilos que hemos usado.

```
N_PROCS=(2 4 6 8 10) # Número de procesos MPI  
N_HILOS_PASOS=(1 2 4 6 8 10) # Número de hilos OpenMP por proceso MPI
```

Éste ha sido el resultado.

P	H	T_EJEC_AVG	FITNESS_AVG	EFICIENCIA_I
2	1	6.2600	4047189.0066	646515.815750
2	2	3.9633	4047142.6966	1021154.769157
2	4	4.4033	4047016.1733	919087.087706
2	6	4.4933	4045594.8200	900361.609507
2	8	4.5000	4045449.8500	898988.855555
2	10	4.5200	4045358.7466	894990.873141
4	1	4.5033	4047830.0500	898858.625896
4	2	3.8400	4046706.8333	1053829.904505
4	4	4.5033	4045361.5600	898310.474540
4	6	5.3266	4049120.9633	760169.895111
4	8	6.1000	4043908.6233	662935.839885
4	10	3.3633	4049553.0900	1204041.593078
6	1	5.5766	4047512.7433	725802.952211
6	2	4.5300	4049383.8133	893903.711545
6	4	5.7633	4047877.7700	702354.166883
6	6	6.9166	4048004.8033	585259.347555
6	8	8.0666	4048860.2400	501928.971313
6	10	4.1200	4047362.8100	982369.614077
8	1	5.6100	4049708.0266	721873.088520
8	2	6.0333	4047450.8466	670851.912982
8	4	7.3433	4046807.9500	551088.468399
8	6	8.9700	4047218.9866	451194.981783
8	8	10.6133	4045658.5833	381187.621503
8	10	5.1100	4045214.7933	791627.161115
10	1	5.9933	4047268.6633	675298.860944
10	2	6.2000	4046351.4133	652637.324725
10	4	8.5200	4046290.1333	474916.682312
10	6	10.5366	4050841.8133	384454.360353
10	8	12.6200	4045059.1000	320527.662440
10	10	5.6566	4046536.8466	715365.563518

Como se puede apreciar en la tabla superior las mejores configuraciones para tiempo, fitness y eficiencia han sido:

- Tiempo: (p=4, h=10), con un tiempo de 3.3633s
- Fitness: (p=10, h=6), con un fitness de 4050841.8133
- Eficiencia: (p=4, h=10), con una eficiencia de 1204041.593078

Por tanto, la mejor configuración ha sido con el binomio p=4, h=10. Si en vez de experimentar con un rango tan amplio de procesos e hilos seguimos explorando con p=4 y aumentamos el número de hilos todo apunta a que el tiempo debería disminuir. Vamos a realizar el experimento.

Tras ejecutar con la siguiente configuración:

```
N_PROCS=(4) # Número de procesos MPI
N_HILOS_PASOS=(10 12 14 16 20) # Número de hilos OpenMP por proceso MPI
```

Éste ha sido el resultado.

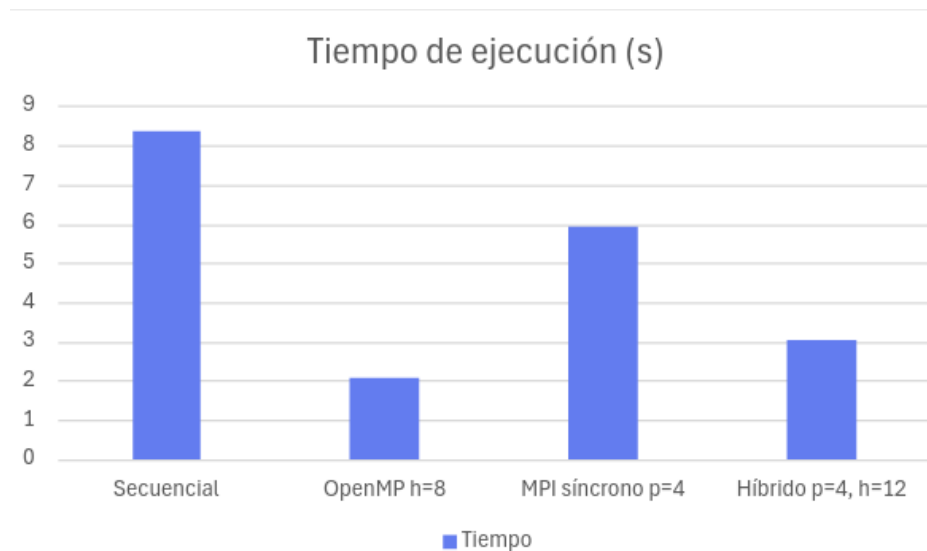
P	H	T_EJEC_AVG	FITNESS_AVG	EFICIENCIA_I
4	10	3.1933	4045345.1133	1266822.758055
4	12	3.0333	4047064.4233	1334211.724293
4	14	3.0466	4045286.2733	1327803.542736
4	16	3.0900	4048678.1866	1310251.840323
4	20	3.0966	4049699.1866	1307788.925466

Con 20 hilos ya deja de mejorar e incluso empeora, por lo que ese ha sido el límite superior.

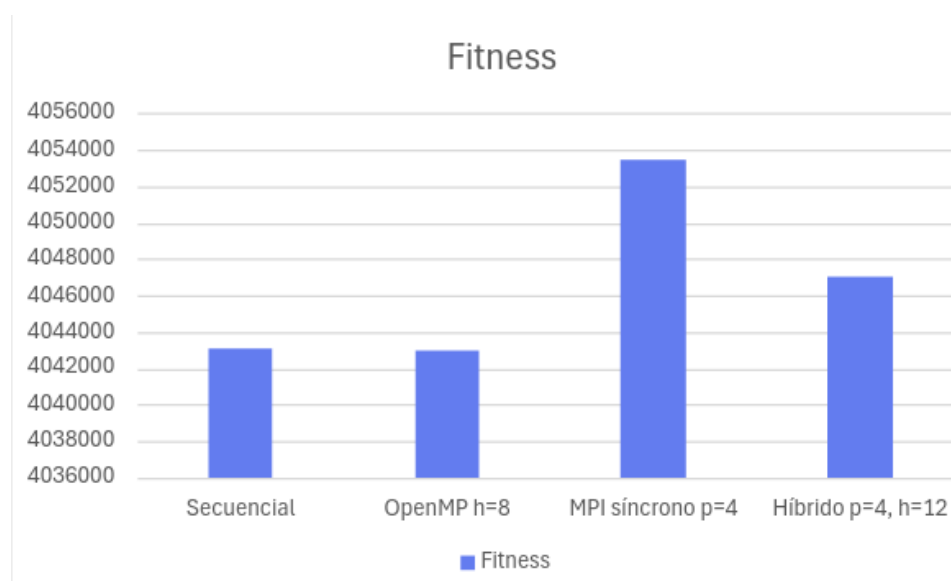
En conclusión, la **mejor configuración** de procesos e hilos para la **versión híbrida** es **p=4** y **h=12** como pone de manifiesto la tabla de la imagen superior.

Si ahora recopilamos en una hoja de cálculo todos los datos de la versión secuencial así como de las óptimas de OpenMP, MPI e Híbrida obtenemos la siguiente tabla:

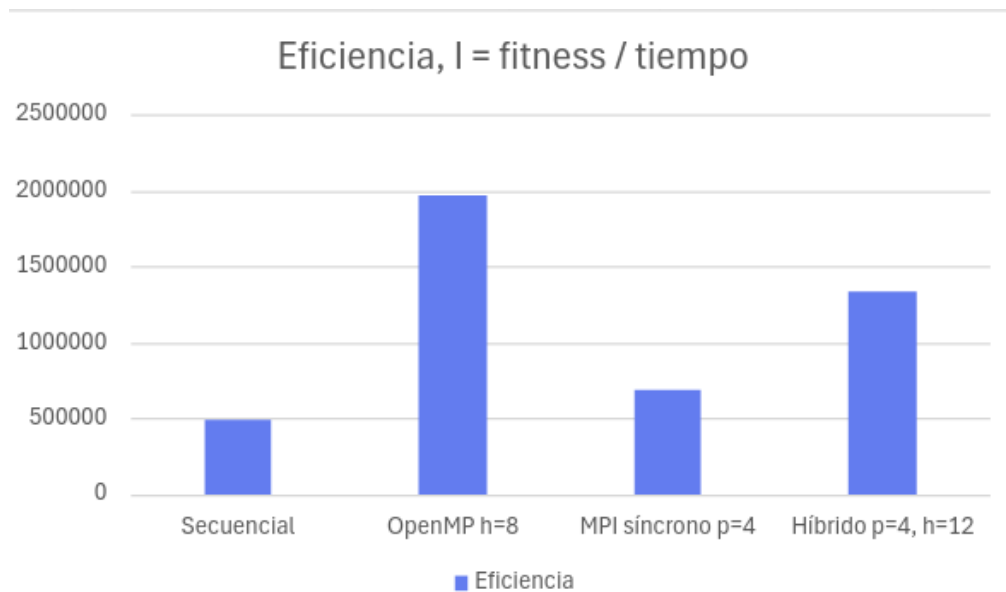
	Tiempo	Fitness	Eficiencia
Secuencial	8,365	4043063	483331
OpenMP h=8	2,055	4042993	1967393
MPI síncrono	5,935	4053417	682968
Híbrido p=4, h=12	3,0333	4047064	1334211



En cuanto al tiempo de ejecución, OpenMP permanece imbatible. En torno a 1 segundo de diferencia respecto a la versión híbrida, que se lleva el segundo puesto. Es evidente que la puesta en marcha de procesos y a su vez hilos en cada proceso conlleva un coste adicional que no ocurre en OpenMP, por lo que en términos de tiempo el algoritmo en memoria compartida es muy eficiente.



Por otro lado, el mejor fitness se ha obtenido con la versión síncrona de MPI y el segundo puesto es de nuevo para la implementación híbrida. Ésto pone de manifiesto la importancia del desarrollo de la población de cada proceso en paralelo en sus respectivas islas así como de las migraciones para obtener una mayor diversidad genética y converger hacia un individuo que arroje una mejor solución, es decir, un mejor fitness.



Finalmente podemos concluir que **en términos de eficiencia y por tanto en términos globales, la mejor implementación** del algoritmo evolutivo es la de **OpenMP**. Su tiempo de ejecución es tan competente que aunque el fitness de las versiones MPI e híbrida sea mejor, no pueden batir al algoritmo paralelo en memoria compartida. Ésto es un claro ejemplo de que en ocasiones, la implementación paralela más simple es la más eficiente y no por añadir complejidad p.ej. con las islas se va a obtener un mejor resultado. Además, por la propia naturaleza del algoritmo evolutivo, lo más costoso es calcular el fitness de cada individuo así como el cruce. Y en ésto, OpenMP brilla.

Cuestión 3

3.1 Obtención de datos experimentales

a) Paraleliza el bucle for correspondiente a la generación inicial de individuos (FOR_INI) que incluye el cálculo del fitness para cada individuo dentro del mismo bucle for.

En primer lugar

```
static unsigned int seed;
#pragma omp threadprivate(seed)

int aleatorio(int n)
{
    //return rand() % n; // genera un número aleatorio entre 0 y n-1
    // genera un número aleatorio entre 0 y n-1 (estado por hilo)
    return rand_r(&seed) % n;
}
```

Y luego en aplicar_mh

```
int maxT = omp_get_max_threads();
#pragma omp parallel num_threads(maxT)
{
    // Cada hilo inicializa un seed en paralelo de forma independiente
    unsigned int local_seed = (unsigned int)time(NULL) + omp_get_thread_num(); // TID, id del hilo
    seed = local_seed;
}

Individuo **poblacion = (Individuo **)malloc(tam_pob * sizeof(Individuo *));
assert(poblacion);

#ifdef TIMEGenPobIni
    double ti = mseconds();
#endif

#pragma omp parallel for num_threads(n_hilos_ini)
for (int i = 0; i < tam_pob; i++)
{
    poblacion[i] = (Individuo *)malloc(sizeof(Individuo));
    poblacion[i]->array_int = crear_individuo(n, m);
    fitness(d, poblacion[i], n, m);
}

#ifdef TIMEGenPobIni
    double tf = mseconds();
    printf("Tiempo bucle for_ini: %.2lf sec\n", (tf - ti)/1000);
#endif
```

b) Modifica el código de la Práctica 1 para que el programa reciba dos nuevos parámetros: el número de hilos (N_HILOS_INI) para el bucle FOR_INI y el número de hilos (N_HILOS_FIT) para el bucle for (FOR_FIT) dentro de la función que realiza el cálculo del fitness.

En el fichero main.c añadimos dos parámetros enteros a la función externa:

```
extern double aplicar_mh(const double *, int, int, int, int, double, int *, int, int);
```

También incluimos los dos nuevos parámetros que serán leídos de la entrada

```
int main(int argc, char **argv)
{
    // Check Number of Input Args
    if(argc < 8) {
        fprintf(stderr, "Ayuda:\n");
        fprintf(stderr, " ./programa n m nGen tamPob m_rate n_hilos_ini n_hilos_fit\n");
        return(EXIT_FAILURE);
    }

    int n = atoi(argv[1]);
    int m = atoi(argv[2]);
    int n_gen = atoi(argv[3]);
    int tam_pob = atoi(argv[4]);
    double m_rate = atof(argv[5]);
    int n_hilos_ini = atoi(argv[6]);
    int n_hilos_fit = atoi(argv[7]);
```

así como la llamada a la función aplicar_mh que se realiza desde el main

```
double value = aplicar_mh(d, n, m, n_gen, tam_pob, m_rate, sol, n_hilos_ini, n_hilos_fit);
```

En mh.c también debemos añadir esos dos parámetros

```
double aplicar_mh(const double *d, int n, int m, int n_gen, int tam_pob, double m_rate, int *sol, int
n_hilos_ini, int n_hilos_fit)
```

y actualizar la declaración en mh.h.

```
double aplicar_mh(const double *, int, int, int, int, double, int *, int, int);
```

c) Instrumentaliza el código para que permita medir el tiempo de ejecución del bucle FOR_INI al variar el número de hilos (por ejemplo, desde 1 hasta 20, con incrementos de 2: 1, 2, 4, 6, 8, ...). En este caso no hay que paralelizar el bucle FOR_FIT.

Para ello debemos añadir el tiempo inicial justo antes del bucle, el tiempo final inmediatamente después y calcular la diferencia.

```

#ifdef TIMEGenPobIni
    double ti = mseconds();
#endif

#pragma omp parallel for num_threads(n_hilos_ini)
for (int i = 0; i < tam_pob; i++)
{
    poblacion[i] = (Individuo *)malloc(sizeof(Individuo));
    poblacion[i]->array_int = crear_individuo(n, m);
    fitness(d, poblacion[i], n, m);
}

#ifdef TIMEGenPobIni
    double tf = mseconds();
    printf("Tiempo bucle for_ini: %.2lf sec\n", (tf - ti)/1000);
#endif

```

En utilidades se ha añadido también esta función que se nos daba en el main.c por defecto para permitir medir el tiempo.

```

static double mseconds() {
    struct timeval t;
    gettimeofday(&t, NULL);
    return t.tv_sec*1000 + t.tv_usec/1000;
}

```

d) Aplicar el algoritmo sobre una instancia del problema de tamaño $n = 30000$ y $m = 28000$ (u otros valores que consideres oportunos para tu sistema computacional de manera que, para una correcta visualización del gráfico, el tiempo de ejecución con 1 hilo se sitúe en el intervalo de 5 a 10 segundos). Fija el valor del parámetro NE en la ecuación 1, por ejemplo, $NE = 6$ (igual al número de cores si el sistema es hexacore) y el número de iteraciones del algoritmo a 0 para que sólo se ejecute la parte de generación inicial de la población (para experimentar con la metodología presentada sólo nos interesa esta función, aunque el proceso real se ampliaría a todas las rutinas del algoritmo). Lanza 5 series de ejecuciones y toma la media de los tiempos experimentales obtenidos con cada número de hilos.

Para que el tiempo de ejecución con 1 hilo se encuentre entre 5 y 10 segundos hemos optado por $n = 12000$ y $m = 10000$. El makefile quedaría así:

```

CC = gcc
CFLAGS = -Wall -O3 -std=c99 -g

omp: main.c mh.c io.c
$(CC) $(CFLAGS) main.c mh.c io.c -lm -fopenmp -o omp -DTIMEGenPobIni

run_omp:
./omp 12000 10000 0 8 0.0 1 1 >> ../output/out
./omp 12000 10000 0 8 0.0 2 1 >> ../output/out
./omp 12000 10000 0 8 0.0 4 1 >> ../output/out
./omp 12000 10000 0 8 0.0 6 1 >> ../output/out
./omp 12000 10000 0 8 0.0 8 1 >> ../output/out
./omp 12000 10000 0 8 0.0 10 1 >> ../output/out
./omp 12000 10000 0 8 0.0 12 1 >> ../output/out
./omp 12000 10000 0 8 0.0 14 1 >> ../output/out
./omp 12000 10000 0 8 0.0 16 1 >> ../output/out
./omp 12000 10000 0 8 0.0 18 1 >> ../output/out
./omp 12000 10000 0 8 0.0 20 1 >> ../output/out

clean:
rm -f omp

```

Como nuestro sistema computacional es de 8 cores, fijamos NE a 8.

A continuación se muestra la tabla tras lanzar 5 series de ejecuciones y tomar la media de los tiempos experimentales obtenidos con cada número de hilos.

<u>h</u>	1	2	3	4	5	media
1	8,76	7,54	9,15	7,2	9,39	8,41
2	4,15	4,79	4,95	4,92	4,81	4,72
4	3,04	2,87	2,85	2,75	2,81	2,86
6	2,88	2,87	2,9	2,92	2,83	2,88
8	1,96	2,01	2,02	2,02	1,96	1,99
10	2,32	2,01	1,96	1,93	1,94	2,03
12	1,94	1,95	2,04	1,93	1,93	1,96
14	1,96	2,01	2,38	2,02	1,94	2,06
16	1,96	1,99	2,02	1,96	1,96	1,98
18	1,96	1,94	1,95	1,94	1,94	1,95
20	1,96	1,99	1,94	2,05	1,95	1,98

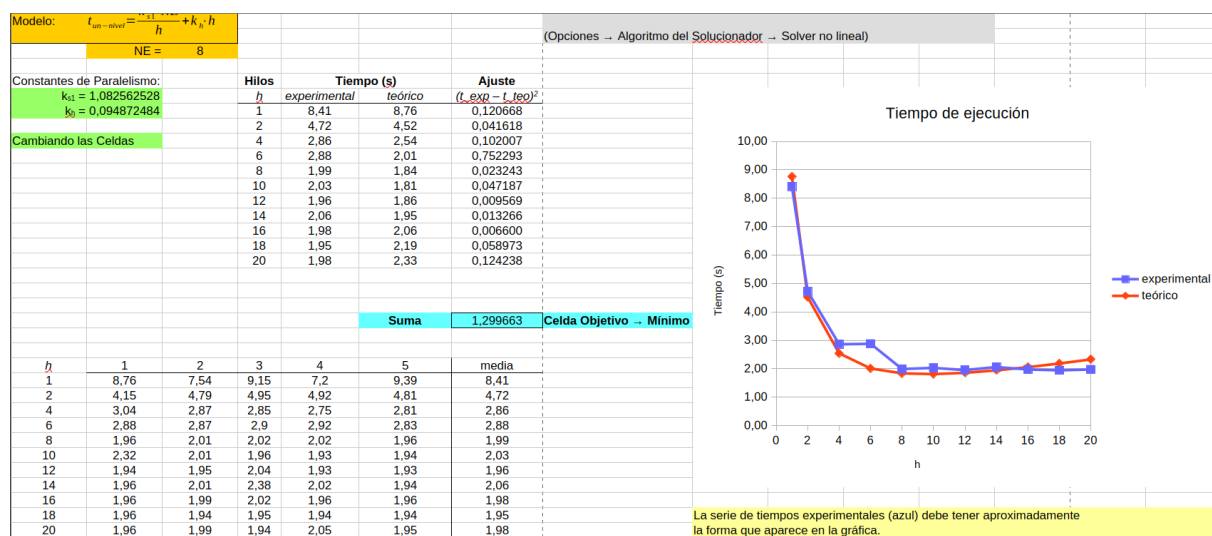
3.2 Instalación de la Rutina (bucle for) con 1 Nivel de Paralelismo

a) Analiza la hoja de cálculo “Modelo 1-Nivel Paralelismo” del documento datos/solver.ods. En ella se presentan tiempos experimentales ($t_{\text{experimental}}$) frente al número de hilos, h , así como los correspondientes al modelo teórico ($t_{\text{teórico}}$) dado por la ecuación 1. Obviamente, debes reemplazar estos datos por tus tiempos obtenidos en el Apartado 3.1.

La tabla del apartado anterior se corresponde con lo solicitado.

b) Instalar la Rutina con 1 Nivel de Paralelismo (obtener valores de constantes del sistema) a partir de los datos experimentales obtenidos en el Apartado 3.1.

Podemos observar en la imagen como el valor de $ks1$ es aproximadamente 1,0826 y el de kh 0,0949.



3.3 Ejecución

a) Despeja el número de hilos óptimo (h_{opt}) de la ecuación 3 en función de NE , $ks1$ y kh , donde los valores de $ks1$ y kh han sido obtenidos en la instalación (Apartado 3.2)

$$\frac{-ks1 \cdot NE}{h^2} + kh = 0 \rightarrow \frac{-ks1 \cdot NE}{h^2} = -kh \rightarrow -ks1 \cdot NE = -kh \cdot h^2$$

$$\rightarrow h = \sqrt{\frac{ks1 \cdot NE}{kh}} = \sqrt{\frac{1'0826 \cdot 8}{0'0949}} = 9'5544$$

Podemos comprobar como el número de hilos óptimo teórico, 9.6, se asemeja a nuestro sistema computacional de 8 cores y al resultado obtenido experimentalmente, pues es a partir de 8 hilos cuando comienza a aplanarse la función del tiempo experimental sin que haya una mejora sustancial.

b) Calcula el tiempo de ejecución que se obtendría con varios valores de h (p.ej.: 3, 5, 7, ...) distintos a los usados en la obtención del modelo en los Apartados 3.1 y 3.2.

$$t = \frac{k_{s1} \cdot NE}{h} + k_h \cdot h$$

$$h = 3: \left[t = \frac{1'0826 \cdot 8}{3} + 0'0949 \cdot 3 = \underline{3'1716 s} \right]$$

$$h = 5: \left[t = \frac{1'0826 \cdot 8}{5} + 0'0949 \cdot 5 = \underline{2'2067 s} \right]$$

$$h = 7: \left[t = \frac{1'0826 \cdot 8}{7} + 0'0949 \cdot 7 = \underline{1'9016 s} \right]$$

$$h = 9: \left[t = \frac{1'0826 \cdot 8}{9} + 0'0949 \cdot 9 = \underline{1'8164 s} \right]$$

$$h = 11: \left[t = \frac{1'0826 \cdot 8}{11} + 0'0949 \cdot 11 = \underline{1'8212 s} \right]$$

$$h = 13: \left[t = \frac{1'0826 \cdot 8}{13} + 0'0949 \cdot 13 = \underline{1'8999 s} \right]$$

$$h = 15: \left[t = \frac{1'0826 \cdot 8}{15} + 0'0949 \cdot 15 = \underline{2'0009 s} \right]$$

$$h = 17: \left[t = \frac{1'0826 \cdot 8}{17} + 0'0949 \cdot 17 = \underline{2'1228 s} \right]$$

$$h = 19: \left[t = \frac{1'0826 \cdot 8}{19} + 0'0949 \cdot 19 = \underline{2'2589 s} \right]$$

c) Realiza las correspondientes ejecuciones para ese número de hilos con NE = 6 (o el valor que hayas elegido para tu sistema) y comprueba la bondad del ajuste del modelo a los datos experimentales.

Lanzaremos 5 ejecuciones con 3, 5, 7,...,19 hilos y tomaremos la media para que la comparación con el tiempo teórico sea más fiable.

```

CFLAGS = -Wall -O3 -std=c99 -g

omp: main.c mh.c io.c
    $(CC) $(CFLAGS) main.c mh.c io.c -lm -fopenmp -o omp -DTIMEGenPobIni

run_omp:
    ./omp 12000 10000 0 8 0.0 3 1 >> ../output/out
    ./omp 12000 10000 0 8 0.0 5 1 >> ../output/out
    ./omp 12000 10000 0 8 0.0 7 1 >> ../output/out
    ./omp 12000 10000 0 8 0.0 9 1 >> ../output/out
    ./omp 12000 10000 0 8 0.0 11 1 >> ../output/out
    ./omp 12000 10000 0 8 0.0 13 1 >> ../output/out
    ./omp 12000 10000 0 8 0.0 15 1 >> ../output/out
    ./omp 12000 10000 0 8 0.0 17 1 >> ../output/out
    ./omp 12000 10000 0 8 0.0 19 1 >> ../output/out

clean:
    rm -f omp

```

3	3,86	3,63	3,89	3,85	3,62	3,77
5	2,87	2,95	2,82	2,75	2,83	2,84
7	3,01	3,16	2,56	2,99	2,94	2,93
9	2,01	2,06	2,01	2,15	1,98	2,04
11	1,96	2,24	1,96	1,98	1,94	2,02
13	1,94	2,17	1,93	1,93	1,96	1,99
15	2,00	2,04	2	1,98	2,16	2,04
17	1,99	1,96	1,97	1,99	1,99	1,98
19	1,98	2,07	2,06	2,04	1,95	2,02

El ajuste del modelo de tiempos es bastante bueno ya que no hay mucha diferencia entre los tiempos teóricos y los experimentales.

d) Varía el tamaño de la población dando valores a NE (por ejemplo 3, 10 u otros que estimes oportunos) y obtén el número de hilos óptimo (hopt) en estos casos. De esta forma, al variar el tamaño del problema no necesitamos volver a determinar experimentalmente el número de hilos óptimo, sino que podemos hacer el cálculo directamente gracias al modelo de tiempos.

$$h_{opt} = \sqrt{\frac{k_{s1} \cdot NE}{k_h}}$$

$$NE = 3; \quad h_{opt} = \sqrt{\frac{1'0826 \cdot 3}{0'0949}} = 5'8582$$

$$NE = 10; \quad h_{opt} = \sqrt{\frac{1'0826 \cdot 10}{0'0949}} = 10'6807$$

e) Ejecuta de nuevo la rutina FOR_INI con $NE = \{3, 10\}$ para obtener experimentalmente h_{opt} en cada caso y compararlos con los respectivos valores arrojados por el modelo. ¿Son similares? ¿Ajusta bien el modelo?

El makefile con $NE = 3$ quedaría así:

```
CC = gcc
CFLAGS = -Wall -O3 -std=c99 -g

omp: main.c mh.c io.c
    $(CC) $(CFLAGS) main.c mh.c io.c -lm -fopenmp -o omp -DTIMEGenPobIni

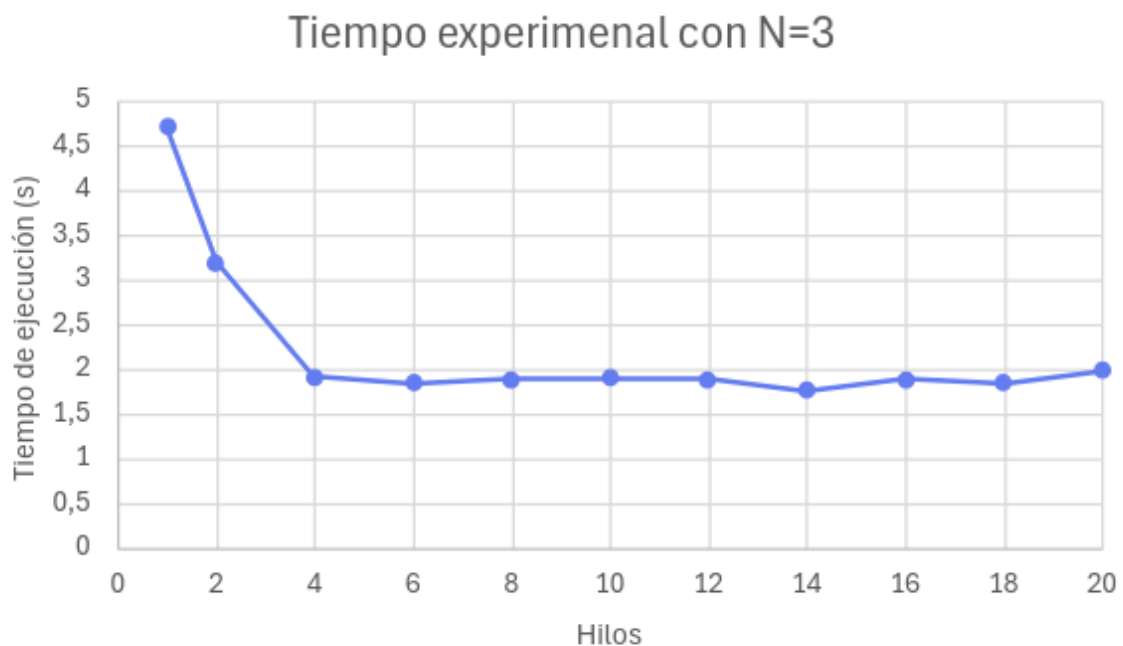
run_omp:
    ./omp 12000 10000 0 3 0.0 1 1 >> ../output/out
    ./omp 12000 10000 0 3 0.0 2 1 >> ../output/out
    ./omp 12000 10000 0 3 0.0 4 1 >> ../output/out
    ./omp 12000 10000 0 3 0.0 6 1 >> ../output/out
    ./omp 12000 10000 0 3 0.0 8 1 >> ../output/out
    ./omp 12000 10000 0 3 0.0 10 1 >> ../output/out
    ./omp 12000 10000 0 3 0.0 12 1 >> ../output/out
    ./omp 12000 10000 0 3 0.0 14 1 >> ../output/out
    ./omp 12000 10000 0 3 0.0 16 1 >> ../output/out
    ./omp 12000 10000 0 3 0.0 18 1 >> ../output/out
    ./omp 12000 10000 0 3 0.0 20 1 >> ../output/out

clean:
    rm -f omp
```

La siguiente tabla muestra la media de los tiempos experimentales tras lanzar 5 ejecuciones con $NE = 3$.

h	1	2	3	4	5	media
1	4,51	4,54	5,32	4,58	4,58	4,71
2	3,41	3,38	3,26	2,81	3,14	3,20
4	1,92	2,01	1,95	1,84	1,85	1,91
6	1,92	1,9	1,92	1,75	1,78	1,85
8	1,88	1,88	1,79	1,74	2,14	1,89
10	1,93	1,84	1,99	1,82	1,91	1,90
12	1,86	1,75	2,13	1,96	1,77	1,89
14	1,77	1,66	2,06	1,72	1,6	1,76
16	1,78	2,28	1,8	1,72	1,84	1,88
18	2,06	1,97	1,59	1,85	1,72	1,84
20	2,33	1,98	1,78	1,94	1,88	1,98

El número de hilos óptimo teórico con $NE = 3$ es 5.8582. Como corrobora la tabla, el modelo ajusta bien porque a partir de 6 hilos no disminuye el tiempo significativamente. El mejor tiempo ha sido de 1.76s con 14 hilos, apenas una décima de segundo menos respecto a los 1.85s con 6 hilos. La gráfica de abajo quizás es más visual.



Por otro lado, así quedaría nuestro makefile con $NE = 10$.

```

CC = gcc
CFLAGS = -Wall -O3 -std=c99 -g

omp: main.c mh.c io.c
    $(CC) $(CFLAGS) main.c mh.c io.c -lm -fopenmp -o omp -DTIMEGenPobIni

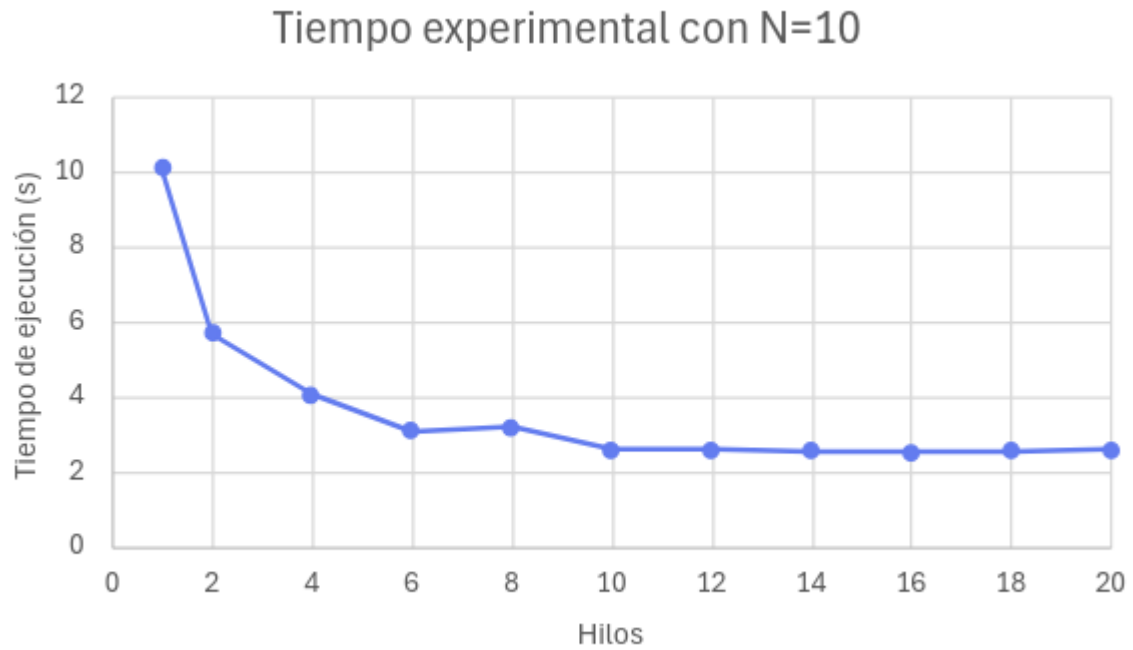
run_omp:
    ./omp 12000 10000 0 10 0.0 1 1 >> ../output/out
    ./omp 12000 10000 0 10 0.0 2 1 >> ../output/out
    ./omp 12000 10000 0 10 0.0 4 1 >> ../output/out
    ./omp 12000 10000 0 10 0.0 6 1 >> ../output/out
    ./omp 12000 10000 0 10 0.0 8 1 >> ../output/out
    ./omp 12000 10000 0 10 0.0 10 1 >> ../output/out
    ./omp 12000 10000 0 10 0.0 12 1 >> ../output/out
    ./omp 12000 10000 0 10 0.0 14 1 >> ../output/out
    ./omp 12000 10000 0 10 0.0 16 1 >> ../output/out
    ./omp 12000 10000 0 10 0.0 18 1 >> ../output/out
    ./omp 12000 10000 0 10 0.0 20 1 >> ../output/out

clean:
    rm -f omp

```

La siguiente tabla muestra la media de los tiempos experimentales tras lanzar 5 ejecuciones con $NE = 10$.

h	1	2	3	4	5	media
1	9,08	10,94	8,91	10,5	11,18	10,12
2	4,78	5,87	5,48	6,33	5,95	5,68
4	4,01	4,05	4,04	4,1	4,08	4,06
6	3,12	3,11	3,12	3,08	3,02	3,09
8	3,25	3,26	3,25	3,22	2,91	3,18
10	2,55	2,63	2,55	2,65	2,58	2,59
12	2,52	2,67	2,62	2,54	2,56	2,58
14	2,58	2,54	2,58	2,63	2,46	2,56
16	2,51	2,49	2,48	2,51	2,57	2,51
18	2,52	2,62	2,48	2,61	2,5	2,55
20	2,87	2,48	2,56	2,52	2,53	2,59



El número de hilos óptimo teórico con $NE = 10$ es de 10.6807. La gráfica pone de manifiesto que ocurre exactamente lo mismo que con $NE = 3$. A partir de 10 hilos la función del tiempo experimental se aplanan, por lo que podemos concluir que la bondad del modelo es muy competente.

Cuestión 4. Modelo de Tiempo en Memoria Compartida con 2 niveles de paralelismo

En este caso hay que paralelizar los bucles FOR_INI y FOR_FIT. Recuerda hacer uso de la función `omp_set_nested(1)` que permite habilitar el uso de paralelismo anidado en OpenMP.

4.1 Instalar la Rutina con 2 Niveles de Paralelismo, es decir, obtener los valores de las constantes del sistema ks_2 , kh_1 y kh_2 en la ecuación 2, donde se ha introducido el nuevo parámetro $|B| = m$ que es el tamaño del individuo. Se deben obtener datos experimentales usando la configuración de parámetros del Apartado 3.1(d), variando el número de hilos en ambos niveles de paralelismo (h_1 , h_2) como se sugiere en la hoja de cálculo “Modelo 2-Niveles Paralelismo” del documento `solver.ods`.

Para poder instalar la rutina debemos realizar los siguientes cambios en el código:

Paralelizar la función `fitness` usando `reduction`, añadir como parámetro `n_hilos_fit` e indicar el número de hilos a usar en la sintaxis del `pragma` con `num_threads(n_hilos_fit)`.

```
void fitness(const double *d, Individuo *individuo, int n, int m, int n_hilos_fit)
{
    double suma = 0.0;

    #pragma omp parallel for num_threads(n_hilos_fit) reduction(+:suma) default(none) shared(d,individuo,n,m)
    for (int i = 0; i < m; i++)
    {
        int elem_i = individuo->array_int[i];
        for (int j = i + 1; j < m; j++)
        {
            int elem_j = individuo->array_int[j];
            suma += distancia_ij(d, elem_i, elem_j, n);
        }
    }

    individuo->fitness = suma;
}
```

Modificar en `mh.h` la declaración de la función `fitness` para añadir el parámetro `n_hilos_fit`.

Añadir `omp_set_nested(1)` al inicio de la función `aplicar_mh` así como `n_hilos_fit` en la llamada a la función `fitness`.

```
double aplicar_mh(const double *d, int n, int m, int n_gen, int tam_pob, double m_rate, int *sol, int n_hilos_ini, int n_hilos_fit)
{
    omp_set_nested(1);

    int maxT = omp_get_max_threads();
    #pragma omp parallel num_threads(maxT)
    {
        // Cada hilo inicializa un seed en paralelo de forma independiente
        unsigned int local_seed = (unsigned int)time(NULL) + omp_get_thread_num(); // TID, id del hilo
        seed = local_seed;
    }

    Individuo **poblacion = (Individuo **)malloc(tam_pob * sizeof(Individuo *));
    assert(poblacion);

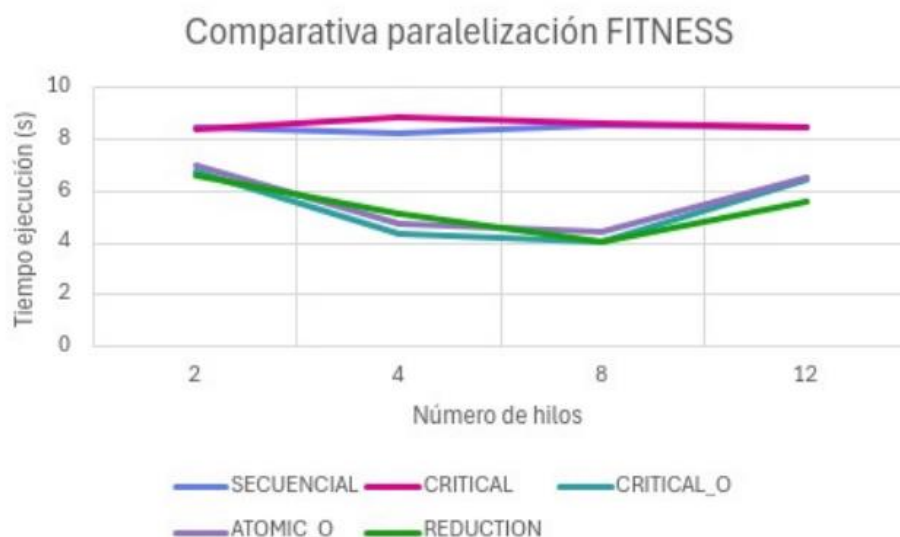
    #ifdef TIMEGenPobIni
        double ti = mseconds();
    #endif

    #pragma omp parallel for num_threads(n_hilos_ini)
    for (int i = 0; i < tam_pob; i++)
    {
        poblacion[i] = (Individuo *)malloc(sizeof(Individuo));
        poblacion[i]->array_int = crear_individuo(n, m);
        fitness(d, poblacion[i], n, m, n_hilos_fit);
    }

    #ifdef TIMEGenPobIni
        double tf = mseconds();
        printf("Tiempo bucle for_ini: %.2lf sec\n", (tf - ti)/1000);
    #endif
}
```

Según la observación 3 del enunciado “considerar el estudio previo realizado sobre el bucle del fitness con reduction de la Práctica 1: **¿merecía la pena poner en marcha muchos hilos para este bucle?** Si la respuesta es negativa, entonces procede variar h2 solo en un intervalo reducido, con la idea de que el modelo de 2-niveles sea lo más preciso posible abarcando todo el dominio de h1, h2.”

En nuestro caso el mejor tiempo con reduction se obtenía con 8 hilos como bien muestra la gráfica de abajo extraída de la práctica 1.



Por tanto, los valores de h2 comprenderán entre 1 y 6, ya que sí que merece la pena en nuestro caso particular poner en marcha muchos hilos, en torno al número de cores físicos. El makefile quedaría de la siguiente forma:

```
omp: main.c mh.c io.c
    $(CC) $(CFLAGS) main.c mh.c io.c -lm -fopenmp -o omp -DIMEGenPobIni

run_omp:
    ./omp 12000 10000 0 8 0.0 1 1 >> ../output/out
    ./omp 12000 10000 0 8 0.0 1 2 >> ../output/out
    ./omp 12000 10000 0 8 0.0 1 4 >> ../output/out
    ./omp 12000 10000 0 8 0.0 1 6 >> ../output/out
    ./omp 12000 10000 0 8 0.0 2 1 >> ../output/out
    ./omp 12000 10000 0 8 0.0 2 2 >> ../output/out
    ./omp 12000 10000 0 8 0.0 2 4 >> ../output/out
    ./omp 12000 10000 0 8 0.0 2 6 >> ../output/out
    ./omp 12000 10000 0 8 0.0 4 1 >> ../output/out
    ./omp 12000 10000 0 8 0.0 4 2 >> ../output/out
    ./omp 12000 10000 0 8 0.0 4 4 >> ../output/out
    ./omp 12000 10000 0 8 0.0 4 6 >> ../output/out
    ./omp 12000 10000 0 8 0.0 6 1 >> ../output/out
    ./omp 12000 10000 0 8 0.0 6 2 >> ../output/out
    ./omp 12000 10000 0 8 0.0 6 4 >> ../output/out
    ./omp 12000 10000 0 8 0.0 6 6 >> ../output/out

clean:
    rm -f omp
```

Una vez lanzadas 5 ejecuciones ésta es la tabla con los tiempos experimentales y su media.

h1	h2	1	2	3	4	5	media
1	1	8,48	9,32	7,89	8,11	8,75	8,51
2	1	6,21	6,55	5,93	5,74	5,6	6,006
4	1	4,38	4,17	4	3,51	4,48	4,108
6	1	3,19	3,17	3,11	3,29	2,87	3,126
1	2	4,8	4,85	4,23	4,93	4,75	4,712
2	2	3,57	3,8	3,42	3,75	3,63	3,634
4	2	2,52	2,4	2,36	2,52	2,44	2,448
6	2	2,34	2,26	2,33	2,24	2,29	2,292
1	4	2,83	2,9	2,78	2,71	2,9	2,824
2	4	3,05	2,25	2,35	2,19	2,36	2,44
4	4	2,2	2,14	2,08	2,03	2,47	2,184
6	4	2,36	2,04	2,05	2,06	2,09	2,12
1	6	2,84	2,83	2,86	2,83	2,83	2,838
2	6	2,48	2,53	2,25	2,54	2,55	2,47
4	6	2,09	2,08	2,12	2,2	2,15	2,128
6	6	2,18	2,2	2,08	2,05	2,05	2,112

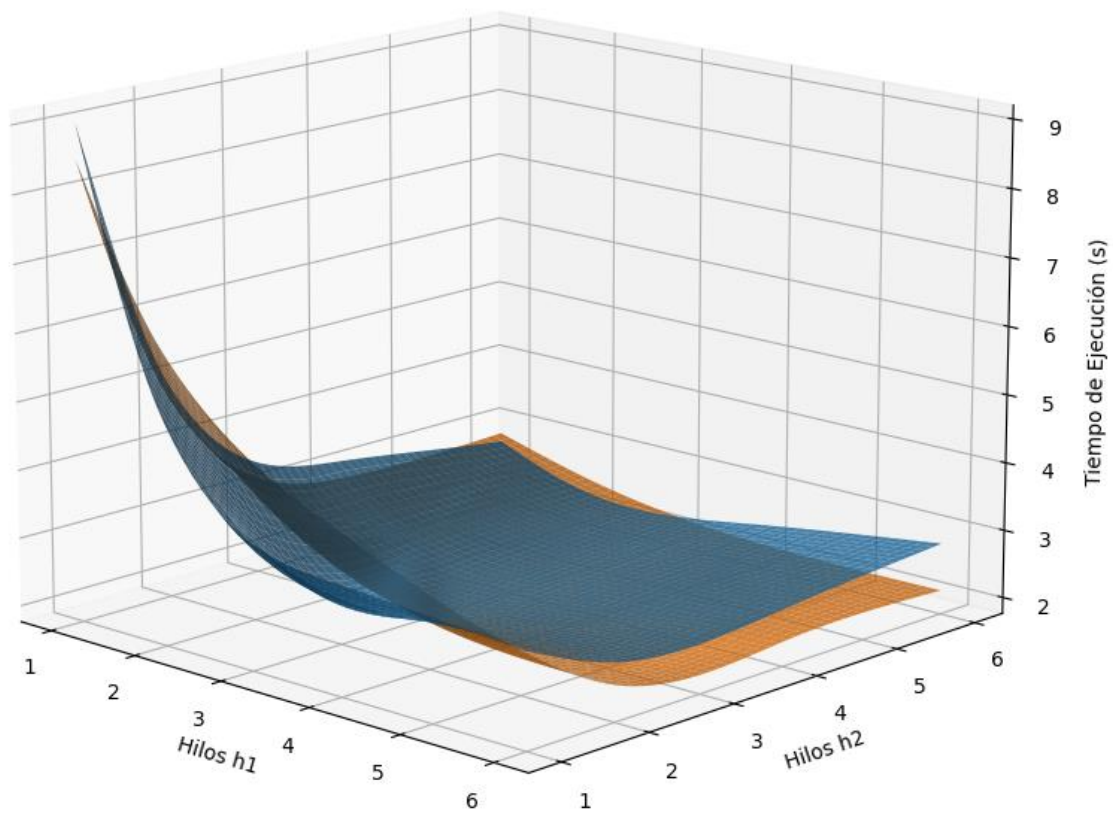
La siguiente imagen muestra la instalación de la rutina para obtener valores de las constantes del sistema a partir de los datos experimentales mostrados anteriormente.

Modelo de tiempo para Función con 2 Niveles de Paralelismo en Memoria Compartida						
Modelo:		$t_{dos-niveles} = \frac{k_{s2} \cdot NE \cdot B }{h_1 \cdot h_2} + k_{h,1} \cdot h_1 + k_{h,2} \cdot h_2$				
		NE = 8				
		B = 10000				
Constantes de Paralelismo:		Hilos		Tiempo		Ajuste
$k_{s2} = 0,0001077464$		$h1$	$h2$	$experimental$	$teórico$	$(t_{exp} - t_{teo})^2$
$k_{h,1} = 0,258928512$		1	1	8,51	9,05	0,288907
$k_{h,2} = 0,1688621214$		2	1	6,006	5,00	1,018941
		4	1	4,108	3,36	0,560247
Cambiando las Celdas		6	1	3,126	3,16	0,001092
		1	2	4,712	4,91	0,037833
		2	2	3,634	3,01	0,388741
		4	2	2,448	2,45	0,000008
		6	2	2,292	2,61	0,100873
		1	4	2,824	3,09	0,070386
		2	4	2,44	2,27	0,028639
		4	4	2,184	2,25	0,004342
		6	4	2,12	2,59	0,219187
		1	6	2,838	2,71	0,016713
		2	6	2,47	2,25	0,048691
		4	6	2,128	2,41	0,078423
		6	6	2,112	2,81	0,481886

Si comparamos los tiempos experimentales con los teóricos podemos confirmar que el modelo ajusta correctamente pese a lo que se comenta en la observación 3 del boletín: “como el rango de hilos con el que se experimenta es muy grande, el modelo trata de ajustar en todos los sectores del dominio, pero a costa de un ajuste regular o malo en general.”

A continuación se muestra una gráfica generada con python que muestra de manera más visual esta comparación.

Comparación de Rendimiento: Experimental vs Teórico



4.2 Despeja el número de hilos óptimo $h_{1,opt}$ y $h_{2,opt}$ de las ecuaciones 4 y 5 en función de NE , $|B|$, ks_2 , kh_1 y kh_2 , donde las constantes han sido obtenidas en la instalación realizada en el apartado anterior.

$$\begin{aligned} \frac{-K_{s2} \cdot NE \cdot |B|}{h_1^2 \cdot h_2} + K_{h1} &= 0; \quad \frac{-K_{s2} \cdot NE \cdot |B|}{h_1^2 \cdot h_2} = -K_{h1}; \\ -K_{s2} \cdot NE \cdot |B| &= -K_{h1} \cdot (h_1^2 \cdot h_2); \quad \boxed{h_1^2 \cdot h_2 = \frac{K_{s2} \cdot NE \cdot |B|}{K_{h1}}} = \\ &= \frac{1'0774 \cdot 10^{-4} \cdot 8 \cdot 10000}{0'2589} = \boxed{33'2916} \\ \boxed{h_1 \cdot h_2^2} &= \frac{K_{s2} \cdot NE \cdot |B|}{K_{h2}} = \frac{1'0774 \cdot 10^{-4} \cdot 8 \cdot 10000}{0'1689} = \boxed{51'0314} \\ \begin{cases} h_1^2 \cdot h_2 = 33'2916 \\ h_1 \cdot h_2^2 = 51'0314 \end{cases} &\rightarrow \begin{aligned} h_2 &= \frac{33'2916}{h_1^2} \\ h_1 &= \frac{51'0314}{h_2^2} = \frac{51'0314 \cdot h_1^4}{33'2916^2} \end{aligned} \\ \frac{h_1}{h_1^4} &= 0'046 \rightarrow \frac{1}{h_1^3} = 0'046 \rightarrow \boxed{h_1 = \sqrt[3]{\frac{1}{0'046}}} = \\ &= \boxed{2'79} \\ \boxed{h_2} &= \frac{33'2916}{2'79^2} = \boxed{4'2769} \end{aligned}$$

4.3 Finalmente, resuelve las siguientes cuestiones:

a) Calcula el tiempo de ejecución que se esperaría para varios valores de h_1 y h_2 (distintos a los usados para instalar el modelo) con la configuración de parámetros utilizada en 4.1.

(4.3)

$$t_{\text{dos-niveles}} = \frac{k_{s2} \cdot NE \cdot |B|}{h_1 \cdot h_2} + k_{h,1} \cdot h_1 + k_{h,2} \cdot h_2$$

a)

$h_1 = 3$

- $h_2 = 3 \rightarrow t = 2'241s$
- $h_2 = 5 \rightarrow t = 2'1958s$
- $h_2 = 7 \rightarrow t = 2'3694s$

$h_1 = 5$

- $h_2 = 3 \rightarrow t = 2'3758s$
- $h_2 = 5 \rightarrow t = 2'4838s$
- $h_2 = 7 \rightarrow t = 2'723s$

$h_1 = 7$

- $h_2 = 3 \rightarrow t = 2'7294s$
- $h_2 = 5 \rightarrow t = 2'9031s$
- $h_2 = 7 \rightarrow t = 3'1705s$

$h_1 = 9$

- $h_2 = 3 \rightarrow t = 3'1560s$
- $h_2 = 5 \rightarrow t = 3'3661s$
- $h_2 = 7 \rightarrow t = 3'6492s$

Por agilidad, hemos obviado escribir a mano para cada una de las configuraciones de hilos la sustitución en la fórmula de los valores apropiados. Dicho esto, el tiempo de ejecución teórico arrojado por el modelo es coherente con $h_{1,opt}$ y $h_{2,opt}$. Sus valores son 2.79 y 4.28 respectivamente. En los tiempos calculados teóricamente observamos como a medida que los valores de h_1 y h_2 se alejan de los valores óptimos, el tiempo de ejecución comienza a aumentar más y más. De hecho, el tiempo más rápido se ha obtenido con $h_1 = 3$ y $h_2 = 5$, muy similar a $h_{1,opt} = 2.79$ y $h_{2,opt} = 4.28$.

b) Varía el tamaño de la población dando valores diferentes a NE y $|B|$ y obtén el número de hilos óptimo $h_{1,opt}$ y $h_{2,opt}$ para cada configuración (no hay que lanzar ejecuciones).

$$\boxed{h_1^2 \cdot h_2 = \frac{K_{s2} \cdot NE \cdot |B|}{K_{h1}}} \quad ; \quad \boxed{h_1 \cdot h_2^2 = \frac{K_{s2} \cdot NE \cdot |B|}{K_{h2}}}$$

$$\boxed{NE = 10 \quad ; \quad |B| = 14000}$$

$$\underline{\underline{h_1^2 \cdot h_2 = \frac{1'0774 \cdot 10^{-4} \cdot 10 \cdot 14000}{0'2589} = 58'2603}}} \quad \parallel \quad \underline{\underline{h_1 \cdot h_2^2 = \frac{1'0774 \cdot 10^{-4} \cdot 10 \cdot 14000}{0'1689} = 89'3049}}}$$

$$\begin{cases} h_1^2 \cdot h_2 = 58'2603 \\ h_1 \cdot h_2^2 = 89'3049 \end{cases} \Rightarrow h_2 = \frac{58'2603}{h_1^2}$$

$$\frac{h_1 \cdot 58'2603^2}{h_1^4} = 89'3049 \rightarrow \frac{1}{h_1^3} = 0'0263 \rightarrow \boxed{h_1 = \sqrt[3]{\frac{1}{0'0263}} = 3'3626}$$

$$\boxed{h_2 = \frac{58'2603}{3'3626} = 17'326}$$

Al aumentar NE a 10 y B a 14000 respecto a los valores de la instalación que fueron NE = 8 y B = 10000 hemos obtenido $h_{1,opt} = 3.36$ y $h_{2,opt} = 17.33$. Como bien se puede apreciar en la fórmula situada al principio de la imagen anterior, h_1 y h_2 son proporcionales a NE y B, por lo que al aumentar estos últimos, también lo hará el número de hilos. De hecho $h_{2,opt}$ se ha disparado de 4.28 en la instalación a 17.33 mientras que $h_{1,opt}$ apenas ha aumentado: 3.36 respecto a 2.79. ¿Por qué está diferencia tan notable entre $h_{1,opt}$ y $h_{2,opt}$? Pues bien, h_2 son los hilos encargados de calcular el fitness de cada individuo de la población, por lo que al aumentar en 4000 el número de elementos de un individuo, está claro que esos hilos tendrán mucho más trabajo.

Cuestión 5. Modelo de Tiempo en Memoria Distribuida

5.1 Añade una nueva hoja de cálculo al documento solver.ods, de forma que permita realizar los cálculos correspondientes a la fase de instalación del modelo de tiempo para paso de mensajes. Comprueba experimentalmente que el tiempo de ordenación tord es realmente despreciable en la ec. 6. Para ello, mide el tiempo de un ciclo completo del algoritmo que incluya computación paralela en las islas, una migración y una ordenación (consultar la Observación (4) para más información).

Para medir el tiempo completo de un ciclo del algoritmo fijaremos NGM a 1 para que se realice una migración.

Para medir el tiempo de ejecución total pondremos el tiempo inicial antes de la llamada al algoritmo evolutivo y el tiempo final después para luego hacer la diferencia.

```
#ifdef TIME
    double ti = 0.0;
    if (myrank == 0) { ti = mseconds(); }
    MPI_Barrier(MPI_COMM_WORLD);
#endif

double value = aplicar_mh(d, n, m, n_gen, tam_pob, m_rate, nem, ngm, sol, myrank, size, mpi_tipo_individuo);

#ifdef TIME
    MPI_Barrier(MPI_COMM_WORLD);
    if (myrank == 0) {
        double tf = mseconds();
        printf("Execution Time: %.2lf sec\n", (tf - ti)/1000);
    }
#endif
```

Mismo modus operandi para medir el tiempo de ordenación que realiza P0 cuando recibe los nem mejores individuos de cada esclavo.

```
// Ordenación
double ti = mseconds();
qsort(buffer_mezcla, total_migrantes, sizeof(Individuo), comp_fitness);
double tf = mseconds();
printf("Tiempo de ordenación: %.2lf s\n", (tf - ti)/1000);
```

Tras lanzar cinco ejecuciones y redondear los tiempos a dos decimales efectivamente podemos demostrar experimentalmente que el tiempo de ordenación es despreciable respecto al tiempo de ejecución que engloba al tiempo de computación y al de comunicaciones.

```
Tiempo de ordenación: 0.00 s
Execution Time: 8.80 sec
Tiempo de ordenación: 0.00 s
Execution Time: 6.81 sec
Tiempo de ordenación: 0.00 s
Execution Time: 8.14 sec
Tiempo de ordenación: 0.00 s
Execution Time: 9.27 sec
Tiempo de ordenación: 0.00 s
Execution Time: 14.86 sec
Tiempo de ordenación: 0.00 s
Execution Time: 19.70 sec
Tiempo de ordenación: 0.00 s
Execution Time: 22.92 sec
```

5.2 Obtén los valores de las constantes del sistema en la ecuación 9 (kcmp, A, B, C y D) al variar el número de procesos con los datos de entrada proporcionados en el Makefile del directorio datos/ejecucion/pm. Consideraremos los siguientes parámetros de entrada: n, m, n_gen, tam_pob.

Para facilitar el proceso de modelado, puedes variar el número de procesos y ajustar los parámetros de entrada como consideres mejor para realizar los experimentos.

Con los parámetros propuestos en el makefile el tiempo de ejecución era demasiado elevado. Por ello, hemos optado por fijar m a 1000 en vez de 2000, NGM a 1 para tomar un ciclo evolutivo en el que se haga una migración y NEM a 10 para que el tiempo de comunicaciones sea notable pero sin excederse demasiado.

Este es el makefile con el que se han lanzado las cinco ejecuciones:

```
CC = mpicc
CFLAGS = -O3 -std=c99 -g -Wall

mpi: main.c mh.c io.c
    $(CC) $(CFLAGS) main.c mh.c io.c -lm -o mpi -DTIME

run_mpi:
    mpirun -np 2 ./mpi 5000 1000 1 500 0.0 1 10 >> ../output/out
    mpirun -np 4 ./mpi 5000 1000 1 500 0.0 1 10 >> ../output/out
    mpirun --oversubscribe -np 6 ./mpi 5000 1000 1 500 0.0 1 10 >> ../output/out
    mpirun --oversubscribe -np 12 ./mpi 5000 1000 1 500 0.0 1 10 >> ../output/out
    mpirun --oversubscribe -np 18 ./mpi 5000 1000 1 500 0.0 1 10 >> ../output/out
    mpirun --oversubscribe -np 24 ./mpi 5000 1000 1 500 0.0 1 10 >> ../output/out
    mpirun --oversubscribe -np 30 ./mpi 5000 1000 1 500 0.0 1 10 >> ../output/out

clean:
    rm -f mpi
```

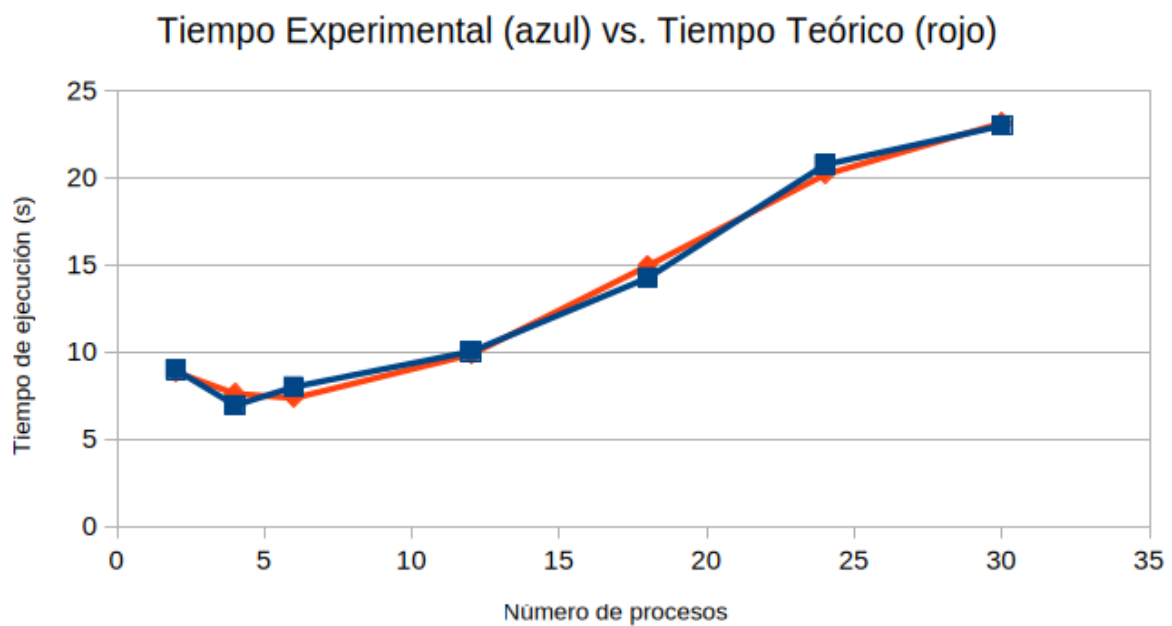
Ésta es la media de los tiempos tras las cinco ejecuciones.

p	1	2	3	4	5	media
2	9,4	8,8	9,02	9,06	8,8	9,016
4	7,11	7,14	6,56	7,26	6,81	6,976
6	8,15	7,63	8,13	8,14	8,14	8,038
12	11,39	9,77	9,04	10,87	9,27	10,068
18	14,27	15,55	13,85	12,99	14,86	14,304
24	20,91	22,08	21,54	19,75	19,7	20,796
30	23,05	22,25	23,52	23,46	22,92	23,04

El valor de las constantes tras aplicar Solver...

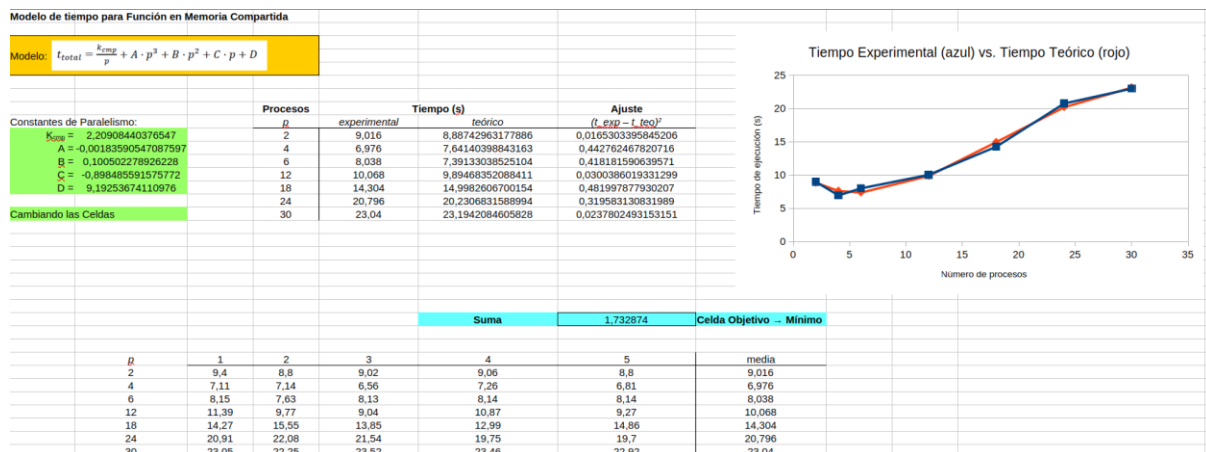
Constantes de Paralelismo:	
K_{comp}	= 2,20908440376547
A	= -0,00183590547087597
B	= 0,100502278926228
C	= -0,898485591575772
D	= 9,19253674110976
Cambiando las Celdas	

... y la comparativa entre el tiempo experimental y teórico una vez ajustado el modelo.



Como ya se concluyó en la práctica 2, el mejor tiempo se obtenía cuando el número de procesos era 4, por lo que añadir más procesos sólo incrementa el overhead.

A continuación se muestra una imagen completa de la hoja de LibreOffice Calc.



5.3 Calcula el tiempo de ejecución teórico para diferentes valores de p y resuelve la ecuación 10 para encontrar popt en función de las constantes del sistema (puedes usar algún método numérico para resolverla y utilizar para ello el propio Solucionador).

Empezaremos calculando el número de procesos óptimo teórico.

$$\frac{-K_{comp}}{p^2} + 3Ap^2 + 2Bp + C = 0$$

$$\frac{-2'2091}{p^2} + 3 \cdot (-0'0018) \cdot p^2 + 2 \cdot 0'1005p + (-0'8985) = 0$$

$$-0'0034p^2 + 0'201p - 0'8985 = \frac{2'2091}{p^2}$$

$$0'0034p^4 - 0'201p^3 + 0'8985p^2 + 2'2091 = 0$$

Hemos usado la herramienta WolframAlpha para resolver la ecuación cuadrática.

Interpretación de la entrada

resuelve

$$0,0054 p^4 - 0,201 p^3 + 0,8985 p^2 + 2,2091 = 0$$

para

p

Resultado

Más dígitos

Formas exactas

$$p \approx 5,6770$$

$$p \approx 32,012$$

$$p \approx -0,2334 - 1,4821 i$$

$$p \approx -0,2334 + 1,4821 i$$

El número de procesos óptimo teórico es 5.7, justo entre el número de procesos óptimo experimental que es cuatro y el número de cores físicos que es ocho.

Si ahora calculamos el tiempo teórico para varios procesos con las constantes obtenidas en la instalación comprobamos como conforme nos alejamos de 5 y 6, los tiempos van incrementándose. De hecho, el mejor tiempo se ha obtenido con $p = 6$, y el número de procesos óptimo teórico es 5.7, por lo que la es totalmente congruente.

$$t_{\text{total}} = \frac{2'2091}{p} - 0'0018 p^3 + 0'1005 p^2 - 0'8985 p + 9'1925$$

$$\begin{aligned}
 p=3 &\rightarrow t=8'0893s \\
 p=5 &\rightarrow t=7'4293s \\
 p=6 &\rightarrow t=7'3989s \\
 p=7 &\rightarrow t=7'5257s \\
 p=10 &\rightarrow t=8'6784s \\
 p=13 &\rightarrow t=10'7118s
 \end{aligned}$$

Cuestión 6. Conclusiones generales y valoración personal

Esta práctica ha sido a la que más tiempo le hemos dedicado sin duda debido a los cálculos teóricos, las gráficas y la toma de datos experimentales que había que realizar. No obstante ha sido gratificante poder tener la comparativa final entre la implementación secuencial, OpenMP, MPI e híbrida del algoritmo evolutivo. Además, creemos que ha sido útil aprender el proceso para generar un modelo de tiempos y comprobar la bondad del ajuste.

En cuanto al reparto del trabajo, como en todas las prácticas, hemos trabajado de forma simultánea apoyándonos el uno en el otro.